

Concurrency Control

Lab1: Experiencing Blocking

The screenshot shows two side-by-side pgcli windows connected to a PostgreSQL 18 database. Both windows are running the same SQL script:

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 4.99  
3 WHERE film_id = 1;  
4  
5  
6  
7 COMMIT;
```

The left window shows the query completing successfully in 78 msec. The right window shows the query being blocked and taking 28 seconds to complete. The status bar at the bottom of the right window indicates: "Total rows: Query complete 00:00:28.676 CRLF Ln 8, Col 8".

When

```
BEGIN;  
UPDATE film SET rental_rate = 4.99  
WHERE film_id = 1;
```

is running, the queries on the right are blocked and have to wait for the queries on the left to commit

Lab2: Replicating Non-repeatable Read

Left Editor Query:

```
1 BEGIN;
2 SELECT rental_rate FROM film
3 WHERE film_id = 10;
4
5
6
7
8 SELECT rental_rate FROM film
9 WHERE film_id = 10;
10 COMMIT;
```

Left Editor Data Output:

rental_rate
4.99

Right Editor Query:

```
1
2
3
4 BEGIN;
5 UPDATE film SET rental_rate = 0.99
6 WHERE film_id = 10;
7 COMMIT;
```

Right Editor Data Output: No data output. Execute a query to get output.

- Result: 4.99

Left Editor Query:

```
1 BEGIN;
2 SELECT rental_rate FROM film
3 WHERE film_id = 10;
4
5
6
7
8 SELECT rental_rate FROM film
9 WHERE film_id = 10;
10 COMMIT;
```

Left Editor Data Output:

rental_rate
0.99

Right Editor Query:

```
1
2
3
4 BEGIN;
5 UPDATE film SET rental_rate = 0.99
6 WHERE film_id = 10;
7 COMMIT;
```

Right Editor Messages:

COMMIT

Query returned successfully in 152 msec.

- Modifies data and commits while the left query is still active
- NEW RESULT: 0.99

Lab3: Creating a Deadlock

Left Window Query:

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 2.99  
3 WHERE film_id = 100;  
4  
5  
6  
7 UPDATE film SET rental_rate = 3.99  
8 WHERE film_id = 200;
```

Right Window Query:

```
1  
2  
3  
4 BEGIN;  
5 UPDATE film SET rental_rate = 2.99  
6 WHERE film_id = 200;  
7  
8  
9 UPDATE film SET rental_rate = 3.99  
10 WHERE film_id = 100;
```

Left Window Messages:

UPDATE 1
Query returned successfully in 66 msec.

Right Window Messages:

UPDATE 1
Query returned successfully in 66 msec.

- The left query is blocked as the right queries are running

Left Window Query:

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 2.99  
3 WHERE film_id = 100;  
4  
5  
6  
7 UPDATE film SET rental_rate = 3.99  
8 WHERE film_id = 200;
```

Right Window Query:

```
1  
2  
3  
4 BEGIN;  
5 UPDATE film SET rental_rate = 2.99  
6 WHERE film_id = 200;  
7  
8  
9 UPDATE film SET rental_rate = 3.99  
10 WHERE film_id = 100;
```

Left Window Messages:

UPDATE 1
Query returned successfully in 1 min 2 secs.

Right Window Messages:

ERROR: deadlock detected
Process 12412 waits for ShareLock on transaction 1026; blocked by process 16416.
Process 16416 waits for ShareLock on transaction 1027; blocked by process 12412.

SQL state: 40P01
Detail: Process 12412 waits for ShareLock on transaction 1026; blocked by process 16416.
Process 16416 waits for ShareLock on transaction 1027; blocked by process 12412.
Hint: See server log for query details.
Context: while locking tuple (132,11) in relation "film"

- When PostgreSQL detect deadlock the queries on the right crash and the one on the left run successfully

Lab4: Preventing Race Conditions (FOR UPDATE)

Left Editor Query:

```
1 BEGIN;
2 SELECT * FROM inventory
3 WHERE inventory_id = 10 FOR UPDATE;
4
5
6
7 INSERT INTO rental
8 (rental_date, inventory_id, customer_id, staff_id)
9 VALUES (NOW(), 10, 1, 1);
10 COMMIT;
```

Right Editor Query:

```
1
2
3
4 BEGIN;
5 SELECT * FROM inventory
6 WHERE inventory_id = 10 FOR UPDATE;
7
8
9
10
11 ROLLBACK;
```

Right Editor Messages:

```
ERROR: current transaction is aborted, commands ignored until end of transaction block
SQL state: 25P02
```

- When the query on the left locks row for update, the right one is blocked immediately

Left Editor Query:

```
1 BEGIN;
2 SELECT * FROM inventory
3 WHERE inventory_id = 10 FOR UPDATE;
4
5
6
7 INSERT INTO rental
8 (rental_date, inventory_id, customer_id, staff_id)
9 VALUES (NOW(), 10, 1, 1);
10 COMMIT;
```

Right Editor Query:

```
1
2
3
4 BEGIN;
5 SELECT * FROM inventory
6 WHERE inventory_id = 10 FOR UPDATE;
7
8
9
10
11 ROLLBACK;
```

Left Editor Messages:

```
COMMIT
Query returned successfully in 617 msec.
```

Right Editor Messages:

```
ROLLBACK
Query returned successfully in 76 msec.
```

- When the queries on the left commit, a ROLLBACK is needed for the queries on the right to run

The image shows two side-by-side PostgreSQL query editors. The left editor contains the following SQL code:

```
1 BEGIN;
2 SELECT * FROM inventory
3 WHERE inventory_id = 10 FOR UPDATE;
4
5
6
7 INSERT INTO rental
8 (rental_date, inventory_id, customer_id, staff_id)
9 VALUES (NOW(), 10, 1, 1);
10 COMMIT;
```

The right editor contains the following SQL code:

```
1
2
3
4 BEGIN;
5 SELECT * FROM inventory
6 WHERE inventory_id = 10 FOR UPDATE;
7
8
9
10
11 ROLLBACK;
```

Below the SQL code, the right editor's 'Data Output' tab shows a successful query result:

Showing rows: 1 to 1 of 1

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
10	2	2	2006-02-15 10:09:17

- Only after a ROLLBACK can the queries on the right run successfully